

Implement and manage semantic models - Power BI Desktop

Lab - DAX Expressions - Creating a measure

Total Customers = COUNT(FactSales[CustomerID])

Total Customers = DISTINCTCOUNT(FactSales[CustomerID])

Sum of Total Amount = SUM(FactSales[TotalAmt])

Lab - DAX Expressions - Aggregation Functions

SalesPerProductID = COUNTROWS(RELATEDTABLE(FactSales))

Lab - DAX Expressions - Filter Functions

Total Amount Color Not Specified = CALCULATE([Sum of Total Amount],FILTER(DimProduct,DimProduct[Color] == "NotSpecified"))

Lab - DAX Expressions - Information Functions

EVALUATE

COLUMNSTATISTICS()

Lab - Power BI Desktop - Building the Date Dimension table

DimDate=CALENDAR(MIN(FactSales[OrderDate]),MAX(FactSales[OrderDate]))

DateYear = FORMAT(DimDate[Date],"YYYY")

DateMonth = FORMAT(DimDate[Date],"MMM")

Lab - Power BI Desktop - Using calculation groups

Year-To-Date = CALCULATE([Sum of Total Amount],DATESYTD(DimDate[Date]))

Month-To-Date = CALCULATE([Sum of Total Amount],DATESMTD(DimDate[Date]))

Year-To-Date = CALCULATE(SELECTEDMEASURE(),DATESYTD(DimDate[Date]))

Month-To-Date = CALCULATE(SELECTEDMEASURE(),DATESMTD(DimDate[Date]))

Implement and manage semantic models - Power BI Desktop

Lab - Microsoft Fabric data warehouse - Ingesting data - COPY command

1. First we can create the table

```
CREATE TABLE [SalesOrderHeader] (  
    [SalesOrderID] [int] NULL,  
    [OrderDate] [date] NULL,  
    [Status] [smallint] NULL,  
    [SalesOrderNumber] [varchar](25) NULL,  
    [CustomerID] [int] NULL,  
    [SalesPersonID] [int] NULL,  
    [TerritoryID] [int] NULL,  
    [SubTotal] [float] NULL,  
    [TaxAmt] [float] NULL,  
    [Freight] [float] NULL,  
    [TotalDue] [float] NULL
```

);

2. The data types in Microsoft Fabric

<https://learn.microsoft.com/en-us/fabric/data-warehouse/data-types>

3. We can then copy the data from the CSV-based file to the table

```
COPY INTO [SalesOrderHeader]
```

```
FROM 'https://appstore5545464.blob.core.windows.net/raw/SalesOrderHeader.csv'
```

```
WITH (
```

```
    FILE_TYPE = 'CSV',
```

```
    FIRSTROW = 2
```

```
);
```

Lab - Microsoft Fabric data warehouse - Ingesting data - COPY command - SAS

1. We can delete the data from the table first

```
DELETE FROM [SalesOrderHeader]
```

2. Then we can copy the data into the table

```
COPY INTO [SalesOrderHeader]
```

```
FROM 'https://appstore5545464.blob.core.windows.net/raw/SalesOrderHeader.csv'
```

```
WITH (
```

```
    FILE_TYPE = 'CSV',
```

```
    CREDENTIAL = (
```

```
        IDENTITY = 'Shared Access Signature'
```

```
        , SECRET = 'sv=2024-11-04&ss=b&srt=sco&sp=rl&se=2025-03-21T16:40:02Z&st=2025-
```

```
03-
```

21T08:40:02Z&spr=https&sig=fw%2BVEPOJp4hnJZ7HjEAmcUxmCfh6FST40BqLJDzQpMw%3D'

),

FIRSTROW = 2

);

Lab - Microsoft Fabric data warehouse - Cloning tables

1. We can create a new schema first

```
CREATE SCHEMA staging;
```

2. Then create a clone of the table

```
CREATE TABLE staging.SalesOrderHeader AS CLONE OF dbo.SalesOrderHeader;
```

3. We can update a value in the SalesOrderHeader table in the staging schema and then verify the results

```
UPDATE staging.SalesOrderHeader SET Status=6
```

```
WHERE SalesOrderID=47350;
```

Lab - Microsoft Fabric warehouse - CREATE TABLE AS SELECT

1. First DROP the table

```
DROP TABLE staging.SalesOrderHeader;
```

2. Then create the table

```
CREATE TABLE staging.SalesOrderHeader
```

```
AS
```

```
SELECT * FROM dbo.SalesOrderHeader;
```

Lab - Microsoft Fabric data warehouse - Building our Dimension tables

1. The following statements can be used to create the dimension tables

```
CREATE TABLE [DimProduct](  
    [ProductID] [int] NULL,  
    [ProductName] [varchar](50) NULL,  
    [MakeFlag] [bit] NULL,  
    [Color] [varchar](15) NULL,  
    [SafetyStockLevel] [smallint] NULL,  
    [ProductModelID] [int] NULL,  
    [ProductModelName] [varchar](500) NULL  
)
```

```
CREATE TABLE [DimCustomer](  
    [CustomerID] [int] NULL,  
    [StoreID] [int] NULL,  
    [StoreName] [varchar](500) NULL  
)
```

Lab - Microsoft Fabric data warehouse - Building the Dimension Date table

1. First create the date dimension table

```
CREATE TABLE [DimDate] (  
    [OrderDate] [date] NULL,  
    [DateYear] [varchar](6) NULL,
```

```
[DateMonth] [varchar](12) NULL,  
[DateMonthNumber] [varchar](6) NULL  
)
```

2. We can create and execute the stored procedure to build the date dimension table

```
CREATE PROC AddDate  
  
AS  
  
BEGIN  
  
    DECLARE @StartDate date;  
  
    DECLARE @EndDate date;  
  
    SET @StartDate='2011-05-31'  
  
    SET @EndDate='2014-06-30'  
  
    WHILE @StartDate <=@EndDate  
  
        BEGIN  
  
            INSERT INTO DimDate  
  
                SELECT  
@StartDate,datepart(yyyy,@StartDate),datename(MONTH,DATEADD(MONTH,datepart(mm,@  
StartDate),-1)),datepart(mm,@StartDate)  
  
                SET @StartDate=dateadd(day,1,@StartDate)  
  
        END  
  
END;  
  
  
EXEC AddDate;
```

Lab - Microsoft Fabric data warehouse - SQL - Basics

1. The below command can be used to get all data from the FactSales table

```
SELECT * FROM FactSales;
```

2. Here we want to get the number of rows from the FactSales table

```
SELECT COUNT(*) FROM FactSales;
```

3. Here we want to extract the orders from the FactSales table where the order was placed between 2011 and 2012

```
SELECT OrderDate,SalesOrderID,OrderQty,TotalAmt
```

```
FROM FactSales
```

```
WHERE YEAR(OrderDate) BETWEEN 2011 AND 2012
```

```
ORDER BY OrderDate
```

Lab - Microsoft Fabric data warehouse - SQL - Group by

1. We can extract the count of sales per Product and also group the results by the ProductID

```
SELECT ProductID,COUNT(SalesOrderID) AS 'SalesCount'
```

```
FROM FactSales
```

```
GROUP BY ProductID
```

```
ORDER BY 'SalesCount'
```

2. We can filter the grouped results so that we only get the sales where the SalesOrderID count is greater than 100

```
SELECT ProductID,COUNT(SalesOrderID) AS 'SalesCount'
```

```
FROM FactSales
```

```
GROUP BY ProductID
```

```
HAVING COUNT(SalesOrderID) > 100
```

```
ORDER BY 'SalesCount'
```

3. We can also combine the use of GROUP BY and the HAVING clause

```
SELECT ProductID,COUNT(SalesOrderID) AS 'SalesCount'  
  
FROM FactSales  
  
WHERE YEAR(OrderDate)=2012  
  
GROUP BY ProductID  
  
HAVING COUNT(SalesOrderID) > 100  
  
ORDER BY COUNT(SalesOrderID)
```

Lab - Microsoft Fabric data warehouse - SQL - Finding duplicate values

1. Below is an example to see if there are any duplicate values when it comes to the SalesOrderID

```
SELECT SalesOrderID,COUNT(SalesOrderID) AS 'SalesCount'  
  
FROM FactSales  
  
GROUP BY SalesOrderID  
  
HAVING COUNT(SalesOrderID)>1
```

Lab - Microsoft Fabric data warehouse - SQL- JOIN

1. We can use the JOIN clause to get the information from both the FactSales and the DimProduct table.

```
SELECT dt.ProductModelName,COUNT(SalesOrderID) AS 'SalesCount'  
  
FROM FactSales fs JOIN DimProduct dt  
  
ON fs.ProductID = dt.ProductID  
  
GROUP BY dt.ProductModelName
```

Lab - Microsoft Fabric data warehouse - SQL user-defined function

1. We can define our function first

```
CREATE FUNCTION GetProducts(@ProductModelID INT)
```

```
RETURNS TABLE
```

```
AS
```

```
    RETURN (SELECT ProductID,ProductName,ProductModelName FROM DimProduct WHERE  
ProductModelID=@ProductModelID)
```

2. Then we can create our view

```
CREATE VIEW FactSalesDimProductvw
```

```
AS
```

```
SELECT
```

```
fs.SalesOrderID,fs.OrderDate,fs.TotalAmt,fs.OrderQty,fs.ProductID,dt.ProductModelID
```

```
FROM FactSales fs JOIN DimProduct dt
```

```
ON fs.ProductID=dt.ProductID
```

3. Finally we can get the rows using the OUTER APPLY statement

```
SELECT *
```

```
FROM FactSalesDimProductvw
```

```
OUTER APPLY GetProducts(ProductModelID)
```

Prepare Data - Microsoft Fabric - Lakehouse

Lab - Microsoft Fabric Apache Spark - Loading data into a dataframe

```
filePath="abfss://parquet@datalake2003040.dfs.core.windows.net/SalesOrderHeader.parquet"
```

```
df = spark.read.load(filePath,format="parquet")
```

```
display(df)
```

Lab - Microsoft Fabric Apache Spark - Performing operations on the data frame

```
filtered_df=df.select("SalesOrderID","TerritoryID").groupBy("TerritoryID").count()
```

```
display(filtered_df)
```

Lab - Microsoft Fabric Apache Spark - Saving the data frame

```
df.write.mode("overwrite").format("delta").saveAsTable("SalesOrderHeader")
```

Lab - Microsoft Fabric Apache Spark - Further working with data

```
filePath="Files/SalesOrderHeader.csv"
```

```
df = spark.read.load(filePath,format="csv",header=True,inferSchema=True)
```

```
filePath="Files/SalesOrderHeader.csv"
```

```
df = spark.read.load(filePath,format="csv",header=True)
```

```
from pyspark.sql.functions import col
```

```
from pyspark.sql.types import *
```

```
final_df=df \
```

```

.withColumn("SalesOrderID",col("SalesOrderID").cast(IntegerType())) \

.withColumn("OrderDate",col("OrderDate").cast(DateType())) \

.withColumn("Status",col("Status").cast(IntegerType())) \

.withColumn("SalesOrderNumber",col("SalesOrderNumber").cast(StringType())) \

.withColumn("CustomerID",col("CustomerID").cast(IntegerType())) \

.withColumn("SalesPersonID",col("SalesPersonID").cast(IntegerType())) \

.withColumn("TerritoryID",col("TerritoryID").cast(IntegerType())) \

.withColumn("SubTotal",col("SubTotal").cast(DecimalType())) \

.withColumn("TaxAmt",col("TaxAmt").cast(DecimalType())) \

.withColumn("Freight",col("Freight").cast(DecimalType())) \

.withColumn("TotalDue",col("TotalDue").cast(DecimalType()))

```

```
display(final_df)
```

Lab - Microsoft Fabric - Making sure the date is displayed properly

```
filePath="Files/SalesOrderHeader.csv"
```

```
df = spark.read.load(filePath,format="csv",header=True)
```

```
from pyspark.sql.functions import col
```

```
from pyspark.sql.types import *
```

```
from pyspark.sql.functions import date_format
```

```
final_df=df \
```

```

.withColumn("SalesOrderID",col("SalesOrderID").cast(IntegerType())) \

.withColumn("OrderDate",to_date("OrderDate","M/d/yyyy H:mm")) \

.withColumn("Status",col("Status").cast(IntegerType())) \

```

```

.withColumn("SalesOrderNumber",col("SalesOrderNumber").cast(StringType())) \

.withColumn("CustomerID",col("CustomerID").cast(IntegerType())) \

.withColumn("SalesPersonID",col("SalesPersonID").cast(IntegerType())) \

.withColumn("TerritoryID",col("TerritoryID").cast(IntegerType())) \

.withColumn("SubTotal",col("SubTotal").cast(DecimalType())) \

.withColumn("TaxAmt",col("TaxAmt").cast(DecimalType())) \

.withColumn("Freight",col("Freight").cast(DecimalType())) \

.withColumn("TotalDue",col("TotalDue").cast(DecimalType()))

```

```
display(final_df)
```

Lab - Microsoft Fabric - Detecting NULL values

1. If you just want to detect if a particular column value is NULL. This will filter all rows where the ProductModelID is NULL

```

from pyspark.sql.functions import col

filtered_df=df.filter(col("ProductModelID").isNull())

```

```
display(filtered_df)
```

2. To detect number of NULL values in each column

```

filePath="Files/data/Product.parquet"

df = spark.read.load(filePath,format="parquet")

```

```
from pyspark.sql.functions import col, sum
```

```
column_nulls=df.select(
```

```
[
```

```
sum(col(cols_in_frame).isNull().cast("int")).alias(cols_in_frame)

for cols_in_frame in df.columns

])

display(column_nulls)
```

Lab - Microsoft Fabric - Checking for duplicate rows

1. Check for duplicates based on specific columns

```
filePath="Files/data/SalesOrderDetail.parquet"

df = spark.read.load(filePath,format="parquet")

duplicate_df=df.groupBy("SalesOrderID").count().filter("count > 1")

display(duplicate_df)
```

2. Checking for entire duplicates rows

```
filePath="Files/data/SalesOrderDetail.parquet"

df = spark.read.load(filePath,format="parquet")

duplicate_df=df.groupBy(df.columns).count().filter("count > 1")

display(duplicate_df)
```

Lab - Microsoft Fabric - Building our Fact data frame

```
salesorderheader_filePath="Files/data/SalesOrderHeader.parquet"

salesorderdetail_filePath="Files/data/SalesOrderDetail.parquet"

salesorderheader_df =
spark.read.load(salesorderheader_filePath,format="parquet").alias("sh")

salesorderdetail_df = spark.read.load(salesorderdetail_filePath,format="parquet").alias("sd")
```

```
fact_sales_df_join=salesorderheader_df.join(salesorderdetail_df,on="SalesOrderID",how="leftouter")
```

```
from pyspark.sql.functions import col
```

```
fact_sales_df=fact_sales_df_join.select(
```

```
col("sh.SalesOrderID").alias("SalesOrderID"),"OrderDate","Status", \
```

```
"SalesOrderNumber","CustomerID","SalesPersonID","TerritoryID","SubTotal","TaxAmt","Freight",  
"TotalDue"
```

```
)
```

```
display(fact_sales_df) // Then you can save this as a managed table in the lakehouse. And yes  
we can build Fact and Dimension tables in the lakehouse
```

Lab - Microsoft Fabric - Building our Dimension data frame

```
product_filePath="Files/data/Product.parquet"
```

```
productmodel_filePath="Files/data/ProductModel.parquet"
```

```
product_df = spark.read.load(product_filePath,format="parquet").alias("pt")
```

```
productmodel_df = spark.read.load(productmodel_filePath,format="parquet").alias("pm")
```

```
dim_product_df_join=product_df.join(productmodel_df,on="ProductModelID",how="leftouter")
```

```
from pyspark.sql.functions import col
```

```
dim_product_df=dim_product_df_join.select(
```

```
"ProductID",col("pt.Name").alias("ProductName"),"MakeFlag","Color","SafetyStockLevel","ProductModelID",col("pm.Name").alias("ProductModelName")
```

)

```
display(dim_product_df)
```

```
customer_filePath="Files/data/Customer.parquet"
```

```
store_filePath="Files/data/Store.parquet"
```

```
customer_df = spark.read.load(customer_filePath,format="parquet").alias("ct")
```

```
store_df = spark.read.load(store_filePath,format="parquet").alias("st")
```

```
dim_customer_df_join=customer_df.join(customer_df,on="ct.StoreID=st.BusinessEntityID",how="leftouter")
```

```
from pyspark.sql.functions import col
```

```
dim_customer_df=dim_customer_df_join.select(
```

```
"CustomerID","StoreID",col("st.Name").alias("StoreName")
```

```
)
```

```
display(dim_customer_df)
```

Microsoft Fabric - EventHouse

Lab - Working with KQL queries

1. **Get the top 100 rows from the table**

tripdata

| take 100

2. Return data where the total amount is greater than 100

tripdata

| where total_amount>100

3. Count the number of rows in the table

tripdata

| count

4. We want to summarize the total payments made based on the passenger count

tripdata

| summarize TotalPayment = sum(total_amount) by passenger_count

Maintain a data analytics solution

Lab - Microsoft Fabric - Giving UPDATED permissions to the user

```
GRANT UPDATE ON OBJECT::dbo.SalesOrderHeader TO  
[fabric_usr01@cloudlearning4000gmail.onmicrosoft.com];
```



```
UPDATE SalesOrderHeader
```

```
SET Status=6
```

```
WHERE SalesOrderID=47350
```

Microsoft Fabric security - Data warehouse - column-level security

```
GRANT SELECT ON dbo.SalesOrderDetail(SalesOrderID,OrderQty,ProductID) TO  
[fabric_usr01@cloudlearning4000gmail.onmicrosoft.com];
```

Microsoft Fabric security - Data warehouse - row-level security

1. First we create a table

```
CREATE TABLE [dbo].[CourseOrders]
```

```
(
```

```
    OrderID int,
```

```
    Agent varchar(200),
```

```
    Course varchar(50),
```

```
    Quantity int
```

```
);
```

2. Then we can insert rows into the table

```
INSERT INTO [dbo].[CourseOrders]
```

```
VALUES(1,'fabric_usr01@cloudlearning4000gmail.onmicrosoft.com','AZ-900',5);
```

```
INSERT INTO [dbo].[CourseOrders]
```

```
VALUES(2,'fabric_usr01@cloudlearning4000gmail.onmicrosoft.com','DP-203',4);
```

```
INSERT INTO [dbo].[CourseOrders]
```

```
VALUES(3,'fabric_usr01@cloudlearning4000gmail.onmicrosoft.com','AZ-104',5);
```

```
INSERT INTO [dbo].[CourseOrders]
```

```
VALUES(4,'warehouse_usr_01@techsup4000gmail.onmicrosoft.com','AZ-303',6);
```

```
INSERT INTO [dbo].[CourseOrders]
VALUES(5,'warehouse_usr_01@techsup4000gmail.onmicrosoft.com','AZ-304',7);

INSERT INTO [dbo].[CourseOrders]
VALUES(6,'warehouse_usr_01@techsup4000gmail.onmicrosoft.com','DP-900',8);
```

3. Then we grant the SELECT statement permission to the user

```
GRANT SELECT ON [dbo].[CourseOrders] TO
[fabric_usr01@cloudlearning4000gmail.onmicrosoft.com];
```

4. We create a new schema

```
CREATE SCHEMA Security;
```

5. We create a function as part of the new schema

```
CREATE FUNCTION Security.securitypredicate(@Agent AS nvarchar(200))

    RETURNS TABLE

WITH SCHEMABINDING

AS

    RETURN SELECT 1 AS securitypredicate_result

WHERE @Agent = USER_NAME() OR USER_NAME() =
'fabricadmin@cloudlearning4000gmail.onmicrosoft.com';
```

6. We create a security policy filter

```
CREATE SECURITY POLICY Filter

ADD FILTER PREDICATE Security.securitypredicate(Agent)

ON [dbo].[CourseOrders]

WITH (STATE = ON);

GO
```

7. We grant access to the user to the security policy filter

```
GRANT SELECT ON Security.securitypredicate TO  
[fabric_usr01@cloudlearning4000gmail.onmicrosoft.com];
```

8. We can confirm via the use of the SELECT statement

```
SELECT * FROM [dbo].[CourseOrders]
```

9. We can then perform a clean-up

```
DROP SECURITY POLICY Filter;
```

```
DROP TABLE [dbo].[CourseOrders];
```

```
DROP FUNCTION Security.securitypredicate;
```

```
DROP SCHEMA Security;
```

Let's work on a Project

Lab - Data warehouse - Building the Date Dimension table

let

```
StartDate = #date(2013,1,1),
```

```
EndDate = #date(2016,05,31),
```

```
NumberOfDays = Duration.Days( EndDate - StartDate ),
```

```
Dates = List.Dates(StartDate, NumberOfDays+1, #duration(1,0,0,0))
```

in

```
Dates
```

```
CREATE TABLE [DimDate] (
```

```
    [InvoiceDate] [date] NULL,
```

```
[Year] [int] NULL,  
[Month] [int] NULL  
)
```

Lab - Data warehouse - SQL - Using the CASE statement

```
SELECT StockItemID, CustomerID, Quantity,  
  
CASE  
  
    WHEN Quantity > 0 AND Quantity < 10 THEN 'Low'  
  
    WHEN Quantity > 10 AND Quantity < 20 THEN 'Medium'  
  
    ELSE 'High'  
  
END AS QuantityTier  
  
FROM FactInvoices
```

Lab - Data warehouse - SQL - Using the PARTITION BY statement

```
1. SELECT  
  
    CustomerID,  
  
    Quantity, ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY Quantity) AS  
    RowNum  
  
FROM FactInvoices;
```

```
2. SELECT  
  
    CustomerID,  
  
    Quantity, SUM(Quantity) OVER (PARTITION BY CustomerID ORDER BY Quantity) AS  
    TotalQuantity  
  
FROM FactInvoices;
```

3. SELECT

CustomerID,

Quantity,InvoiceDate,ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY InvoiceDate DESC) AS RowNum

FROM FactInvoices;

4. WITH CustomerInfo AS

(

SELECT

CustomerID,

Quantity,InvoiceDate,ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY InvoiceDate DESC) AS RowNum

FROM FactInvoices

)

SELECT CustomerID,Quantity,InvoiceDate FROM CustomerInfo WHERE RowNum=1